

Chris Nuhn

COINS 496 Senior Portfolio

Thursday, April 09, 2010

Table of Contents

Projects

1.1	Crogger	COIN 315
1.2	Sailboat	COIN 325
1.3	Gaming Website	COIN 332
1.4	Homework 8	COIN 333

Ethics Papers

2.1	Non-Free Speech	COIN 325
2.2	Microsoft Not Acting Quite So Soft.....	COIN 333
2.3	Declining Violence Hits Video Games	COIN 431

Presentations

3.1	Anagrams	COIN 315
3.2	GCC Optimization	COIN 431

Crogger

COIN 315 – Data Structures
Spring 2009
COIN 315 Grade: B+

Programming Language: C++
Compiled Using: Visual Studio 2008

To run this, use the “Crogger.exe” or upload the files into Visual Studio and run the program.


```
//Student Name      : Chris Nuhn
//Program Name      : Crogger
//Creation Date     : Spring 2009
//Late Modified Date : Spring 2009
//COIN Course       : 315 - Data Structures
//Grade Received    : B+
#ifndef WORLD_H
#define WORLD_H

#include "CConsole.h"

class Player;
class Danger;

const int WIDTH = 40;
const int HEIGHT = 12;
const int DANGERS = 6;

class World
{
public:
    World(int del);
    void run();
    void drawworld();

private:
    int p_delay;
    char p_move;
    Danger *dangers[DANGERS];
    Player *pl;
    CConsole *con;
};

#endif
```

```
#include "World.h"
#include "Player.h"
#include "Danger.h"
#include "Tractor.h"
#include "Car.h"
#include "SemiTruck.h"
#include "FallingLog.h"
#include "Gator.h"
#include "ShortLog.h"

int main()
{
    World w(50);
    w.run();
}

World::World(int del)
{
    p_delay = del;
    con = new CConsole(WIDTH, HEIGHT + 5);
    pl = new Player(3, con);

    int dcount = 0;
    dangers[dcount++] = new Tractor(5, 3, 8, 5, con, pl);
    dangers[dcount++] = new Car(4, 2, 7, 2, con, pl);
    dangers[dcount++] = new SemiTruck(4, 4, 6, 0, con, pl);
    dangers[dcount++] = new FallingLog(4, 6, 4, 3, con, pl);
    dangers[dcount++] = new Gator(5, 3, 3, 1, con, pl);
    dangers[dcount++] = new ShortLog(8, 2, 2, 0, con, pl);
}

void World::run()
{
    con->delay(p_delay);
    drawworld();
    pl->draw();
    con->printString(0, HEIGHT - 1, "Lives:   Level:   Count:   Score:   ");
    con->printInt(7, HEIGHT - 1, pl->getlives());
    con->printInt(16, HEIGHT - 1, pl->getlvl());
    con->printInt(25, HEIGHT - 1, pl->gettoolvl());
    con->printInt(34, HEIGHT - 1, pl->getscore());

    pl->enter();

    while(pl->getlives() > 0)
    {
        for(int i = 0; i < DANGERS; i++)
        {
            dangers[i]->move();
        }
        if(con->getInput(p_move)) {pl->move(p_move);}
        pl->draw();
    }
}

void World::drawworld()
{
    int count = -2;
    for(int i = 0; i < WIDTH; i++)
    {
        con->setColor(white, black);
        con->printChar(i, 0, '#');
        con->printChar(i, 1, '#');
        count++;
    }
}
```

```
    if((count % 8) == 2) {con->printChar(i,1,' ');}
    con->printChar(i,HEIGHT - 2,' ');
    con->setColor(white,brown);
    con->printChar(i,(HEIGHT / 2) - 1,' ');
    con->setColor(white, cyan);
    con->printChar(i,(HEIGHT / 2) - 2,' ');
    con->printChar(i,(HEIGHT / 2) - 3,' ');
    con->printChar(i,(HEIGHT / 2) - 4,' ');
}
for(int i = 0; i < DANGERS; i++)
{
    dangers[i]->draw();
}
```

```
}
```

```
#ifndef TRACTOR_H
#define TRACTOR_H

#include "Danger.h"
#include "CConsole.h"

class Tractor : public Danger
{
public:
    Tractor(int row, int s, int y_cord, int off, CConsole *c, Player *p);
};

#endif
```

```
#include "Tractor.h"
```

```
Tractor::Tractor(int row, int s, int y_cord, int off, CConsole *c, Player *p) : Danger  
    (row, s, y_cord, off, 6, 'T', true, green, black, c, p)  
{  
}
```

```
#ifndef SHORTLOG_H
#define SHORTLOG_H

#include "Danger.h"
#include "CConsole.h"

class ShortLog : public Danger
{
public:
    ShortLog(int row, int s, int y_cord, int off, CConsole *c, Player *p);
};

#endif
```

```
#include "ShortLog.h"
```

```
ShortLog::ShortLog(int row, int s, int y_cord, int off, CConsole *c, Player *p) : Danger  
(row, s, y_cord, off, 3, 'L', true, brown, cyan, c, p)  
{  
}
```

```
#ifndef SEMITRUCK_H
#define SEMITRUCK_H

#include "Danger.h"
#include "CConsole.h"

class SemiTruck : public Danger
{
public:
    SemiTruck(int row, int s, int y_cord, int off, CConsole *c, Player *p);
};

#endif
```

```
#include "SemiTruck.h"
```

```
SemiTruck::SemiTruck(int row, int s, int y_cord, int off, CConsole *c, Player *p) :  
    Danger(row, s, y_cord, off, 3, 'S', true, blue, black, c, p)  
{  
}
```

```
#ifndef PLAYER_H
#define PLAYER_H

#include "CConsole.h"
#include "World.h"

class Player
{
public:
    Player(int p_lives, CConsole *c);
    void draw() const;
    void move(char p_move);
    void reset();
    void leveled();
    void enter();
    void die();
    void top();
    void lvld();
    void upscore(int amount);
    void color();
    bool dead();
    int getlives() const;
    int getscore() const;
    int gettoolvl() const;
    int getlvl() const;
    int gety_cord() const;
    int getx_cord() const;

private:
    int lives, toolvl, lvl, score, x, y;
    bool line_score[HEIGHT];
    Color temp_fg, temp_bg;
    char temp_sym;
    // Danger *dangers[DANGERS];
    CConsole *con;
};

#endif
```

```
#include "Player.h"
#include "World.h"
```

```
Player::Player(int p_lives, CConsole *c)
```

```
{
    lives = p_lives;
    score = 0;
    toolvl = 5;
    lvl = 1;
    con = c;
    line_score[HEIGHT - 3] = true;
    reset();
}
```

```
int Player::getlives() const {return lives;}
int Player::getscore() const {return score;}
int Player::gettoolvl() const {return toolvl;}
int Player::getlvl() const {return lvl;}
int Player::getx_cord() const {return x;}
int Player::gety_cord() const {return y;}
```

```
void Player::draw() const
{
    if(y > HEIGHT / 2 - 1) {con->setColor(white,black);}
    if(y < HEIGHT / 2 - 1) {con->setColor(white,cyan);}
    if(y == HEIGHT / 2 - 1) {con->setColor(white, brown);}
    con->printChar(x,y, '@');
}
```

```
void Player::move(char p_move)
```

```
{
    con->printChar(x,y, ' ');
    switch(p_move)
    {
        case '2': y++; break;
        case '4': x--; break;
        case '6': x++; break;
        case '8': y--; break;
    }

    if(x > WIDTH - 1) {x = 0;}
    if(x < 0) {x += WIDTH;}
    if(y > HEIGHT - 4) {y = HEIGHT - 3;}
    if(y < 2)
    {
        if(!dead()) {leveled();}
        else {reset();}
    }
    if(x == 0 && y == 4)
    {
        die();
    }
    if(line_score[y] == false)
    {
        upscore(5 * lives);
        line_score[y] = true;
    }
}
```

```
void Player::reset()
```

```
{
    x = WIDTH / 2;
    y = HEIGHT - 3;
    for(int i = 0; i < HEIGHT; i++)
    {
        line_score[i] = false;
    }
}
```

```
    }
    line_score[HEIGHT - 3] = true;
    draw();
}

void Player::leveled()
{
    con->setColor(white, black);
    toolvl--;
    if(toolvl < 1)
    {
        lvl++;
        con->printInt(16, HEIGHT - 1, getlvl());
        toolvl = 5;
        lives++;
        con->printInt(7, HEIGHT - 1, getlives());
        lvl();
        upscore(lives * lvl * 50);
    }
    else
    {
        top();
        upscore(lives * lvl * 10);
    }
    con->printInt(25, HEIGHT - 1, gettoolvl());
    reset();
}

bool Player::dead()
{
    if(y == 1)
    {
        if(x == 3 || x == 11 || x == 19 || x == 27 || x == 36)
        {
        }
        else
        {
            con->setColor(white, black);
            con->printInt(7, HEIGHT - 1, getlives() - 1);
            die();
            return true;
        }
    }
    return false;
}

void Player::enter()
{
    con->setColor(white, black);
    con->printString(0, HEIGHT, "Press Enter To Continue");
    con->waitForEnter();
    con->printString(0, HEIGHT, " ");
}

void Player::die()
{
    lives--;
    con->setColor(white, black);
    con->printString(0, HEIGHT, "You have Died");
    con->waitForEnter();
    con->printString(0, HEIGHT, " ");
    con->printInt(7, HEIGHT - 1, getlives());
    reset();
}
```

```
void Player::top()
{
    con->setColor(white, black);
    con->printString(0, HEIGHT, "You Have Reached The Top");
    con->waitForEnter();
    con->printString(0, HEIGHT, "                ");
}

void Player::lvlid()
{
    con->setColor(white, black);
    con->printString(0, HEIGHT, "You Have Increased Levels");
    con->waitForEnter();
    con->printString(0, HEIGHT, "                ");
}

void Player::upscore(int amount)
{
    score += amount;
    con->setColor(white, black);
    con->printInt(34, HEIGHT - 1, getscore());
}
```

```
#ifndef GATOR_H
#define GATOR_H

#include "Danger.h"
#include "CConsole.h"

class Gator : public Danger
{
public:
    Gator(int row, int s, int y_cord, int off, CConsole *c, Player *p);
};

#endif
```

```
#include "Gator.h"
```

```
Gator::Gator(int row, int s, int y_cord, int off, CConsole *c, Player *p) : Danger(row, s  
    , y_cord, off, 4, '=', false, blue, cyan, c, p)  
{  
}
```

```
#ifndef FALLINGLOG_H
#define FALLINGLOG_H

#include "Danger.h"
#include "CConsole.h"

class FallingLog : public Danger
{
public:
    FallingLog(int row, int s, int y_cord, int off, CConsole *c, Player *p);
};

#endif
```

```
#include "FallingLog.h"
```

```
FallingLog::FallingLog(int row, int s, int y_cord, int off, CConsole *c, Player *p) :  
    Danger(row, s, y_cord, off, 3, '=', true, purple, cyan, c, p)  
{  
}
```

```
#ifndef DANGER_H
#define DANGER_H

#include "CConsole.h"
#include "World.h"
#include <string>

class Player;

using namespace std;

class Danger
{
public:
    Danger(int row, int s, int y_cord, int off, int sp, char sy, bool r, Color fg, Color bg, CConsole *c, Player *p);
    void draw();
    void move();
    bool dead();

private:
    bool c_move, c_right;
    int rowcount, size, spaces, y, x, offset, speed, count;
    Color fgcolor, bgcolor;
    string d_sym, sym;
    Player *p1;
    CConsole *con;
};

#endif
```

```
#include "Danger.h"
#include "Player.h"
```

```
Danger::Danger(int row, int s, int y_cord, int off, int sp, char sy, bool r, Color fg,
    Color bg, CConsole *c, Player *p)
{
    rowcount = row;
    size = s;
    y = y_cord;
    x = off;
    offsct = off;
    speed = sp * 2000;
    sym = sy;
    fgcolor = fg;
    bgcolor = bg;
    con = c;
    c_right = r;
    pl = p;

    spaces = (WIDTH + 1 - (rowcount * size)) / rowcount;
    count = 0;

    for(int i = 0; i < rowcount; i++)
    {
        for(int j = 0; j < size; j++)
        {
            if(j == 0 && fgcolor == blue && bgcolor == cyan)
            {
                d_sym.append(">");
            }
            else
            {
                d_sym.append(sym);
            }
        }
        for(int k = 0; k < spaces; k++)
        {
            d_sym.append(" ");
        }
    }
}

void Danger::draw()
{
    con->setColor(fgcolor, bgcolor);
    for(int i = 0; i < WIDTH; i++)
    {
        if((x + i) < WIDTH)
        {
            con->printChar(x + i, y, d_sym[i]);
        }
        else
        {
            con->printChar(x + i - WIDTH, y, d_sym[i]);
        }
    }
}

void Danger::move()
{
    count++;
    if(count == speed)
    {
        if(c_right == true) {x++;}
        else {x--;}
        if(x > WIDTH) {x -= WIDTH;}
    }
}
```

```

    if(x < 0) {x += WIDTH;}
    count = 0;
    draw();
    if(bgcolor == cyan)
    {
        if(y == p1->gety_cord())
        {
            if(c_right)
            {
                p1->move('6');
            }
            else
            {
                p1->move('4');
            }
        }
    }
}
if(dead()) {p1->die();}
}

bool Danger::dead()
{
    if(y == p1->gety_cord())
    {
        if(bgcolor == black)
        {
            if(p1->getx_cord() - x >= 0)
            {
                if(d_sym[p1->getx_cord() - x] == sym[0])
                    return true;
            }
            else
            {
                if(d_sym[p1->getx_cord() - x + WIDTH] == sym[0])
                    return true;
            }
            return false;
        }
        else
        {
            if(p1->getx_cord() - x >= 0)
            {
                if(d_sym[p1->getx_cord() - x] == '6' || d_sym[p1->getx_cord() - x] == '4')
                    return true;
            }
            else
            {
                if(d_sym[p1->getx_cord() - x + WIDTH] == '6' || d_sym[p1->getx_cord() - x + WIDTH] == '4')
                    return true;
            }
            return false;
        }
    }
    return false;
}

```

```
#pragma once
#include <string>
#include <conio.h>
#include <windows.h>
```

```
enum Color {black=0, blue=FOREGROUND_BLUE, green=FOREGROUND_GREEN, red=FOREGROUND_RED,
purple=FOREGROUND_BLUE + FOREGROUND_RED, cyan=FOREGROUND_BLUE+FOREGROUND_GREEN,
brown=FOREGROUND_GREEN+FOREGROUND_RED, white=FOREGROUND_BLUE+FOREGROUND_GREEN+
    FOREGROUND_RED};
```

```
class CConsole {
public:
```

```
    CConsole(int width, int height);
    void setColor(Color fore, Color back) {SetConsoleTextAttribute(m_hConsole, fore +
        (back<<4) );}
    bool getInput(char& ch);
    void waitEnter(); // wait until the user presses ENTER
    void gotoXY(int x, int y); //moves cursor to position x (from left), y (from top)
        //0,0 is upper left corner
    void printChar(int x, int y, char ch); //prints ch at x,y, moves cursor over 1 to
        right
    void printChar(char ch) {_putch(ch);} //prints ch where current cursor is, moves
        cursor over 1 to right
    void printString(int x, int y, const std::string& s);
    void printString(const std::string& s);
    void printStringClearLine(const std::string& s); //clears rest of line after string
        printed
    void printStringClearLine(int x, int y, const std::string& s);
    void printInt(int x, int y, int val); //print an integer value at x,y
    void printInt(int val); //print an integer value
    void delay(int amount) {Sleep(amount);}
```

```
private:
```

```
    HANDLE m_hConsole;
    int m_width;
    int m_height;
```

```
};
```

```

#include "CConsole.h"

void CConsole::waitForEnter() { // wait until the user presses ENTER
    int ch;
    do {
        ch = _getch();
    } while ( ch != '\n' && ch != '\r');
}

/* The getInput function checks whether the user has hit a key and immediately
returns (so it does NOT wait for a key to be pressed). It returns:
    true if the user has hit a key; in this case, the ch variable is set to
        the typed character
    false if the user has not hit any key; in this case, the ch variable
        will be unchanged*/
bool CConsole::getInput(char& ch) {
    if (_kbhit()) { //has key been hit?
        int c = _getch();
        if (c != 0xE0) // first of the two sent by arrow keys, E0 is escape
            ch = c;
        else {
            c = _getch(); //get second char sent by escape sequence from arrows
            switch (c) {
                case 'K': /* left */ ch = '4'; break;
                case 'M': /* right */ ch = '6'; break;
                case 'H': /* up */ ch = '8'; break;
                case 'P': /* down */ ch = '2'; break;
                default: ch = '?'; break; //special character but not arrow
            }
        }
        return true;
    }
    return false;
}

CConsole::CConsole(int width, int height) : m_width(width), m_height(height) {
    m_hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO x;
    GetConsoleCursorInfo(m_hConsole, &x);
    x.bVisible=false;
    SetConsoleCursorInfo(m_hConsole, &x); //hides the cursor
}

void CConsole::gotoXY(int x, int y) { //moves cursor to position x (from left), y (from
    top)
    //0,0 is upper left corner
    if (x < 0 || x >= m_width || y < 0 || y >= m_height) //invalid x,y
        return;
    COORD pos = { x, y }; //same as pos.X=x; pos.Y=y;
    SetConsoleCursorPosition(m_hConsole, pos);
}

void CConsole::printChar(int x, int y, char ch) { //prints ch at x,y, moves cursor over 1
    to right
    gotoXY(x,y);
    printChar(ch);
}

void CConsole::printString(int x, int y, const std::string& s) {
    gotoXY(x,y);
    printString(s);
}

void CConsole::printString(const std::string& s) {
    for (size_t i = 0; i < s.length(); i++)
        printChar(s[i]);
}

```

```
}

void CConsole::printStringClearLine(int x, int y, const std::string& s) { //clears rest of
    line after string printed
    gotoXY(x,y);
    printStringClearLine(s);
}

void CConsole::printStringClearLine(const std::string& s) { //clears rest of line after
    string printed
    printString(s);
    CONSOLE_SCREEN_BUFFER_INFO info;
    GetConsoleScreenBufferInfo(m_hConsole,&info);
    for (int col=info.dwCursorPosition.X; col < m_width; col++)
        printChar(' '); //print spaces for rest of line, console b/c don't know if we
        started at 0
}

void CConsole::printInt(int x, int y, int val) { //print an integer value
    gotoXY(x,y);
    printInt(val);
}

void CConsole::printInt(int val) { //print an integer value
    char buf[12]={0};
    _itoa_s(val, buf, 11, 10); //safely convert int to ascii text
    printString(buf); //constructor for string automatically called for char *
}
```

```
#ifndef CAR_H
#define CAR_H

#include "CConsole.h"
#include "Danger.h"

class Car : public Danger
{
public:
    Car(int row, int s, int y_cord, int off, CConsole *c, Player *p);
};

#endif
```

```
#include "Car.h"
```

```
Car::Car(int row, int s, int y_cord, int off, CConsole *c, Player *p) : Danger(row, s, y_cord, off, 2, 'C', false, red, black, c, p) ✓  
{  
}
```

Sailboat

COIN 325 – Java I
Fall 2008
COIN 325 Grade: A

Programming Language: Java
Compiled Using: Java SDK

To run this program, navigate to the folder containing “Final.java” in the command prompt. Then run using the command “java Final”. The class files are already included.



Start

Stop

20

25

15

40

Set Speeds

Restart



3rd Place

2nd Place

4th Place

1st Place

```
//Student Name      : Chris Nuhn
//Program Name      : Sailboat
//Creation Date     : December 2, 2008
//Late Modified Date : December 2, 2008
//COIN Course       : 325 - Java I
//Grade Received    : A

import java.awt.*;
import javax.swing.*;
import java.awt.event.*;
import java.applet.*;

public class Sailboat extends Applet
{
    Timer animationtimer = new Timer(500, new TimerAction());
    public int x1 = 1;
    public int y1 = 50;
    public int s1 = 1;
    public int x2 = 1;
    public int y2 = 160;
    public int s2 = 2;
    public int x3 = 1;
    public int y3 = 270;
    public int s3 = 3;
    public int x4 = 1;
    public int y4 = 380;
    public int s4 = 4;

    public boolean st;
    public boolean nd;
    public boolean rd;
    public boolean th;
    public int count = 0;
    public String lane1 = "";
    public String lane2 = "";
    public String lane3 = "";
    public String lane4 = "";

    public Button Start;
    public Button Stop;
    public Button SetSpeed;
    public Button Restart;
    public TextField Speed1;
    public TextField Speed2;
    public TextField Speed3;
    public TextField Speed4;

    public ImageIcon black;
    public ImageIcon red;
    public ImageIcon green;
    public ImageIcon blue;

    Panel appletPanel;

    AudioClip bell;

    public void start()
    {

    }

    public void init()
    {
        bell = getAudioClip(getDocumentBase(), "bell.wav");
    }
}
```

```
black = new ImageIcon(getClass().getResource("black.png"));
red = new ImageIcon(getClass().getResource("red.png"));
blue = new ImageIcon(getClass().getResource("blue.png"));
green = new ImageIcon(getClass().getResource("green.png"));
```

```
Start = new Button("Start");
Start.setLocation(5,5);
Start.setSize(40,20);
add(Start);
```

```
Stop = new Button("Stop");
Stop.setLocation(45,5);
Stop.setSize(40,20);
add(Stop);
```

```
Speed1 = new TextField(5);
add(Speed1);
Speed2 = new TextField(5);
add(Speed2);
Speed3 = new TextField(5);
add(Speed3);
Speed4 = new TextField(5);
add(Speed4);
```

```
SetSpeed = new Button("Set Speeds");
add(SetSpeed);
```

```
Restart = new Button("Restart");
add(Restart);
}
```

```
public boolean action(Event e, Object args)
{
    if (e.target == Start)
    {
        if(animationtimer.isRunning() == true)
        {
            animationtimer.restart();
        }
        else
        {
            animationtimer.start();
        }
    }
    else if(e.target == Stop)
    {
        animationtimer.stop();
    }
    else if(e.target == SetSpeed)
    {
        s1 = Integer.parseInt(Speed1.getText());
        if(s1 > 40)
            s1 = 40;
        s2 = Integer.parseInt(Speed2.getText());
        if(s2 > 40)
            s2 = 40;
        s3 = Integer.parseInt(Speed3.getText());
        if(s3 > 40)
            s3 = 40;
        s4 = Integer.parseInt(Speed4.getText());
        if(s4 > 40)
            s4 = 40;
    }
    else if(e.target == Restart)
    {

```

```

        x1 = x2 = x3 = x4 = count = 0;
        animationtimer.stop();
        st = nd = rd = th = false;
        lane1 = lane2 = lane3 = lane4 = "";
    }
    repaint();
    return true;
}

```

```

public void paint(Graphics g)
{
    black.paintIcon(this,g,x1,y1);
    blue.paintIcon(this,g,x2,y2);
    red.paintIcon(this,g,x3,y3);
    green.paintIcon(this,g,x4,y4);

```

```

    g.drawLine(0,490,490,490);
    g.drawLine(490,50,490,490);

```

```

    checkwinner();

```

```

    g.drawString(lane1,500,100);
    g.drawString(lane2,500,200);
    g.drawString(lane3,500,300);
    g.drawString(lane4,500,400);
}

```

```

    public class TimerAction implements ActionListener
    {

```

```

        public void actionPerformed( ActionEvent actionEvent)
        {
            if(x1 > 400 || x2 > 400 || x3 > 400 || x4 > 400)
                bell.play();
            if(x1 > 400)
                x1 = 0;
            if(x2 > 400)
                x2 = 0;
            if(x3 > 400)
                x3 = 0;
            if(x4 > 400)
                x4 = 0;

            x1 += 1 + s1/5;
            x2 += 1 + s2/5;
            x3 += 1 + s3/5;
            x4 += 1 + s4/5;

            repaint();
        }
    }

```

```

public void checkwinner()
{
    if(x1 > 400 || x2 > 400 || x3 > 400 || x4 > 400)
    {
        if(st == false)
        {
            if(x1 > 400)
            {
                if(lane1 == "")
                {
                    lane1 = "1st Place";

```

```
        count++;
    }
}
if(x2 > 400)
{
    if(lane2 == "")
    {
        lane2 = "1st Place";
        count++;
    }
}
if(x3 > 400)
{
    if(lane3 == "")
    {
        lane3 = "1st Place";
        count++;
    }
}
if(x4 > 400)
{
    if(lane4 == "")
    {
        lane4 = "1st Place";
        count++;
    }
}

switch(count)
{
case 4: th = true;
case 3: rd = true;
case 2: nd = true;
case 1: st = true; break;
}
}

if(nd == false)
{
    if(x1 > 400)
    {
        if(lane1 == "")
        {
            lane1 = "2nd Place";
            count++;
        }
    }
    if(x2 > 400)
    {
        if(lane2 == "")
        {
            lane2 = "2nd Place";
            count++;
        }
    }
    if(x3 > 400)
    {
        if(lane3 == "")
        {
            lane3 = "2nd Place";
            count++;
        }
    }
    if(x4 > 400)
    {
        if(lane4 == "")
```

```
        {
            lane1 = "2nd Place";
            count++;
        }
    }

    switch(count)
    {
        case 4: th = true;
        case 3: rd = true;
        case 2: nd = true;
        case 1: st = true; break;
    }
}

if(rd == false)
{
    if(x1 > 400)
    {
        if(lane1 == "")
        {
            lane1 = "3rd Place";
            count++;
        }
    }
    if(x2 > 400)
    {
        if(lane2 == "")
        {
            lane2 = "3rd Place";
            count++;
        }
    }
    if(x3 > 400)
    {
        if(lane3 == "")
        {
            lane3 = "3rd Place";
            count++;
        }
    }
    if(x4 > 400)
    {
        if(lane4 == "")
        {
            lane4 = "3rd Place";
            count++;
        }
    }

    switch(count)
    {
        case 4: th = true;
        case 3: rd = true;
        case 2: nd = true;
        case 1: st = true; break;
    }
}

if(th == false)
{
    if(x1 > 400)
    {
        if(lane1 == "")
        {
            lane1 = "4th Place";
        }
    }
}
```

```
        count++;
    }
}
if(x2 > 400)
{
    if(lane2 == "")
    {
        lane2 = "4th Place";
        count++;
    }
}
if(x3 > 400)
{
    if(lane3 == "")
    {
        lane3 = "4th Place";
        count++;
    }
}
if(x4 > 400)
{
    if(lane4 == "")
    {
        lane4 = "4th Place";
        count++;
    }
}

switch(count)
{
case 4: th = true;
case 3: rd = true;
case 2: nd = true;
case 1: st = true; break;
}
```

}

}

}

}